



Formation Spring

1

IoC par Annotations

Spring et les annotations

2

- ✓ IoC et Annotations
 - ✓ @Autowired
 - ✓ @Value
 - ✓ @Qualifier
 - ✓ @Component, @Service, @Repository
- ✓ Component scan
- ✓ Lever les ambiguïtés

IoC par annotation

- Vous pouvez gérer les liens entre vos beans via
 - Fichier de configuration XML `<beans>...</beans>`
 - Classe de configuration `@Configuration`
- En ce qui concerne **vos beans** 'métiers' (DAOs, Services, Contrôleurs, Autres) il est conseillé de faire usage des annotations
- Les annotations **ne remplacent pas** la configuration, elles la complète

IoC par annotations avec XML



➤ Injection via XML

```
<bean id="transfertService" class="com.service.TransfertServiceImpl">  
  <constructor-arg ref="referentielDao" />  
</bean>  
  
<bean id="referentielDao" class="com.dao.ReferentielDao" />
```

- La déclaration des beans saute et est remplacée par
- `<context:component-scan .../>`

```
<context:component-scan base-package="com.service, com.dao, com.controler" />
```

IoC par annotations avec @Configuration



➤ Injection via @Configuration

```
@Configuration
public class SpringConfiguration {

    @Bean
    public IReferentielDao referentielDao() {
        IReferentielDao resu = new ReferentielDao();
        ...
        return resu;
    }

    @Bean
    public ITransfertServiceImpl transfertService() {
        return new TransfertServiceImpl(this.referentielDao());
    }
}
```

➤ La déclaration des beans métiers saute et est remplacée par

➡ @ComponentScan

```
@Configuration
@ComponentScan(basePackages= {"com.service", "com.dao", "com.controler"})
```

IoC par annotations



➤ Mes classes métiers :

```
package com.service;

@Service("transfertService")
public class TransfertServiceImpl implements ITransfertService {
    private final IReferentielDAO referentiel;

    @Autowired
    public TransfertServiceImpl(@Qualifier("referentielDao") IReferentielDAO pReferentiel) {
        this.referentiel = pReferentiel;
    }
}
```

Exemple IoC via le constructeur

```
package com.dao;

@Repository("referentielDao")
public class ReferentielDao implements IReferentielDao {

    public ReferentielDao() {
        super();
    }
    // insert, update, delete, select ...
}
```

IoC par annotations - JSR-330 (1/2)

- La JSR-330 spécifie le standard en matière d'annotations pour l'injection de dépendances
 - **@Inject**
 - Google et SpringSource mènent conjointement le lead
 - Dernière publication parue en 2009
- Spring implémente la spécification, mais a aussi réalisé sa propre version

IoC par annotations - JSR-330 (2/2)

➤ En JSR-330 (pur Java)

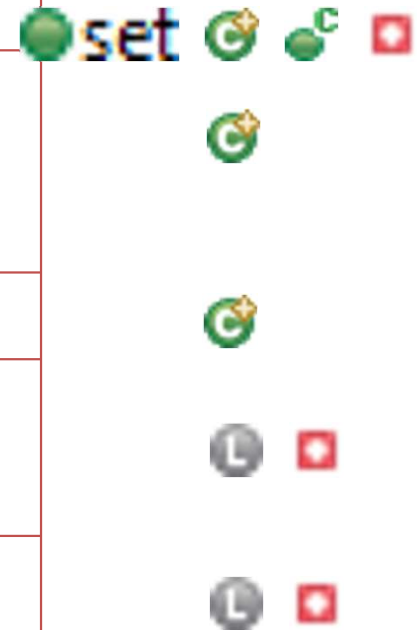
```
package com.fspring;

import javax.inject.Inject;
import javax.inject.Named;

@Named("transfertService")
public class TransfertServiceImpl implements ITransfertService {
    @Inject
    public TransfertServiceImpl(@Named("hibernateDao") IReferentielDAO pReferentiel) {
        this.referentiel = pReferentiel;
    }
}
```

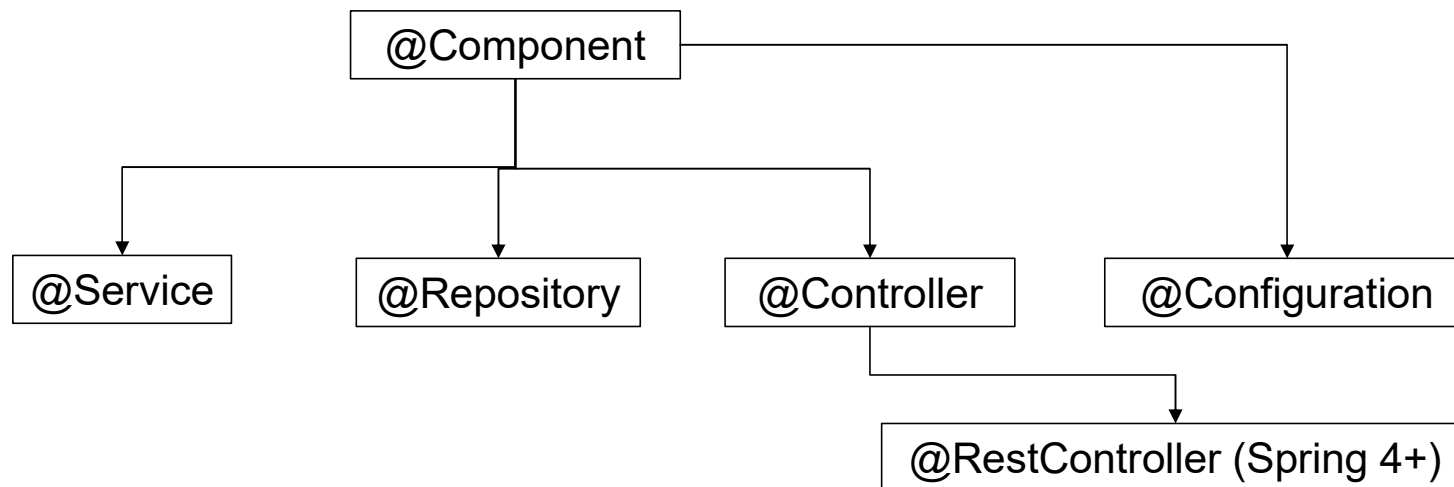

IoC par annotations – En Spring

Spring	Commentaires
@Autowired	Indique qu'il faut réaliser une injection de dépendance Se place sur un attribut, une méthode set, un constructeur
@Component	Indique que cet objet est un bean
@Service	De type Service métier
@Repository	De type DAO
@Controller	De type contrôleur
@RestController	De type contrôleur web service Rest (Spring 4+)
@Scope	Indique le scope du bean Accompagne @Component
@Qualifier	Permet d'indiquer un id de bean Se place sur un attribut, un paramètre de méthode set, un paramètre de constructeur Accompagne @Autowired
@Value	Récupère une valeur de fichier <i>properties</i> chargée par un property-placeholder Se place sur un attribut, un paramètre de méthode set, un paramètre de constructeur
@Required	Indique si le lien doit être obligatoire ou non Accompagne @Autowired
@Lazy	Indique si le bean doit être fait tout de suite ou non Accompagne @Autowired



IoC par annotations – Les stéréotypes

- Un stéréotype permet de caractériser le rôle d'un bean au sein du contexte
- Spring propose en standard un certain nombre de stéréotypes



- Vous pouvez faire usage du stéréotype que vous voulez, mais plus vous êtes précis mieux c'est

Autowiring et scanning des composants

@Autowired

- Modalité d'injection avec l'annotation @Autowired
 - Par défaut obligatoire.
 - Basé sur le type du bean à injecter.
 - Basé sur le nom via l'annotation « @Qualifier ».

@Autowired

```
public TransfertServiceImpl(@Qualifier("referentielDao") IReferentielDAO pReferentiel) {  
    this.referentiel = pReferentiel;  
}
```

- Ici notre exemple porte sur le constructeur de la classe TransfertServiceImpl. Le Spring va injecter dans l'appel du constructeur le bean qui a l'id referentielDao

Autowiring et scanning des composants component-scanning (1/3)

- Toutes les classes annotées avec « @Component » (ou enfant) sont gérées en tant que bean
- Les composants sont détectés au démarrage de l'application

```
<context:component-scan base-package="com.france.fspring" />
```

```
@ComponentScan(basePackages= {"com.france.fspring"})
```

- Indique la racine – les sous-packages sont également scannés.
 - Les archives JAR des dépendances sont prises en compte
- Attention à la désignation du package
 - Un niveau trop élevé impacte le temps de chargement du contexte.
 - Un niveau trop faible risque d'ignorer certaines classes.

Autowiring et scanning des composants component-scanning (2/3)

➤ Très mauvais

```
<context:component-scan base-package="org,com" />
```

```
@ComponentScan(basePackages= {"org", "com"})
```

➤ Correct

```
<context:component-scan base-package="com.france.app" />
```

```
@ComponentScan(basePackages= {"com.france.app"})
```

Autowiring et scanning des composants component-scanning (2/3)

➤ Optimisé

```
<context:component-scan base-package=  
    "com.france.app.dao, com.france.app.service, com.france.app.controller" />
```

```
@ComponentScan(basePackages=  
    {"com.france.app.dao", "com.france.app.service", "com.france.app.controller"})
```

- Il est possible de placer cette déclaration dans plusieurs fichiers ou classes de configuration.
 - ➡ Elle ne sera pas écrasée mais composée

Autowiring et scanning des composants

Exclusion de beans



- Utiliser des filtres pour inclure ou exclure des beans

```
<context:component-scan base-package="com.france.fspring">  
  <context:include-filter type="regex" expression="**Stub,*Repository"/>  
  <context:exclude-filter type="annotation"  
    expression="org.springframework.stereotype.Repository"/>  
</context:component-scan>
```

- Type – indique le type de filtre à appliquer.

- ☐ **annotation** : pour faire référence à une annotation (généralement un stéréotype Spring).
- ☐ **aspectj** : expression de type AspectJ
- ☐ **assignable** : classes qui héritent ou implémentent la classe/interface
- ☐ **custom** : un objet Java qui respecte l'interface `org.springframework.core.type.TypeFilter`
- ☐ **regex** : une expression régulière

- Expression – dépendant du type de filtre.

- Utile pour exclure le(s) package(s) selon l'environnement cible

Autowiring et scanning des composants

Exclusion de beans



- Vous pouvez faire de même avec les classes de configuration via les deux attributs de l'annotation `@ComponentScan`
 - `includeFilters`
 - `excludeFilters`
- Ils prennent des tableaux de type **`ComponentScan.Filter`**
- Chacun d'eux a un type `org.springframework.context.annotation.FilterType`
 - ❑ ***ANNOTATION*** : pour faire référence à une annotation (généralement un stéréotype Spring).
 - ❑ ***ASPECTJ*** : expression de type AspectJ
 - ❑ ***ASSIGNABLE_TYPE*** : classes qui héritent ou implémentent la classe/interface
 - ❑ ***CUSTOM*** : un objet Java qui respecte l'interface `org.springframework.core.type.TypeFilter`
 - ❑ ***REGEX*** : une expression régulière

```
@ComponentScan(basePackages = { "com.france.fspring" },  
    includeFilters = {@ComponentScan.Filter(type=FilterType.REGEX, pattern="*Stub,*Repository") },  
    excludeFilters = {@ComponentScan.Filter(type=FilterType.ANNOTATION,  
                                            classes= org.springframework.stereotype.Repository.class) })
```


Autowiring et scanning des composants usage @Autowired

➤ Injection par constructeur

```
@Autowired  
public TransfertServiceImpl(IReferentielDAO pReferentiel) {  
    this.referentiel = pReferentiel;  
}
```

L'attribut *referentiel* est
souvent déclaré **final**

➤ Injection par méthode setXxx

- Le préfixe « set » n'est pas obligatoire
- Spring appellera la méthode

```
@Autowired  
public void setReferentiel(IReferentielDAO pReferentiel) {  
    this.referentiel = pReferentiel;  
}
```

Autowiring et scanning des composants

usage @Autowired

➤ Injection par membre / attribut

➤ Plus besoin du setXxx

@Autowired

```
private IReferentielDAO referentiel;
```

Autowiring et scanning des composants

Gestion des ambiguïtés (1/3)

➤ Que se passe-t-il dans le cas suivant ?

@Service

```
public class TransfertServiceImpl implements ITransfertService {  
    @Autowired  
    public TransfertServiceImpl(IReferentielDAO pReferentiel) {  
        this.referentiel = pReferentiel;  
    }  
}
```

@Repository

```
public class ReferentielParticulierImpl implements IReferentielDAO {...}
```

@Repository

```
public class ReferentielEntrepriseImpl implements IReferentielDAO {...}
```

- Spring ne sait pas lequel de ces deux beans DAO il doit injecter.
- Une exception est levée

```
org.springframework.beans.factory.NoSuchBeanDefinitionException: No unique bean of type  
[com.france.fspring.dao.IReferentielDAO] is defined: expected single matching bean but found 2:  
[referentielParticulierImpl, referentielEntrepriseImpl]
```

Autowiring et scanning des composants

Gestion des ambiguïtés (2/3)

- L'annotation « **@Primary** » indique à Spring le bean à injecter par défaut.

```
@Repository
@Primary
public class ReferentielParticulierImpl implements IReferentielDAO {...}
```

```
@Repository
public class ReferentielEntrepriseImpl implements IReferentielDAO {...}
```

```
@Service
public class TransfertServiceImpl implements ITransfertService {
    @Autowired
    public TransfertServiceImpl(IReferentielDAO pReferentiel) {
        this.referentiel = pReferentiel;
    }
}
```

- Ici, c'est bien un **ReferentielParticulierImpl** qui sera injecté dans le constructeur

Autowiring et scanning des composants

Gestion des ambiguïtés (3/3)

- L'annotation « **@Qualifier** » permet de résoudre l'ambiguïté

```
@Repository("referencielParticulier")  
public class ReferencielParticulierImpl implements IReferencielDAO {...}
```

```
@Repository("referencielEntreprise")  
public class ReferencielEntrepriseImpl implements IReferencielDAO {...}
```

```
@Service  
public class TransfertServiceImpl implements ITransfertService {  
    @Autowired  
    public TransfertServiceImpl(@Qualifier("referencielParticulier") IReferencielDAO pReferenciel) {  
        this.referenciel = pReferenciel;  
    }  
}
```

- Ici, c'est bien un **ReferencielParticulierImpl** qui sera injecté dans le constructeur

Autowiring et scanning des composants

Les noms des composants

- Si les noms des composants ne sont pas spécifiés
 - Les noms sont automatiquement générés.
 - La règle est **le nom court de classe avec la première lettre en minuscule.**
- Les bonnes pratiques
 - Si possible **ne pas préciser les noms** des composants.
 - Il est rare de définir plusieurs implémentations pour un même bean au sein d'un contexte applicatif.
 - Dans le cas où l'on fait usage de Mock, il suffit de faire usage de **Profile** ou de plusieurs classes/fichiers de configuration Spring

@Value

- Vous pouvez utiliser l'annotation **@Value** sur
 - Un attribut
 - Un paramètre de méthode
- Elle prend toujours une chaîne de caractères qui représente une clef d'un fichier de propriétés chargé dans le contexte Spring

@Value

➤ Exemple :

@Repository

```
public class ReferentielParticulierImpl implements IReferentielDAO {...}
```

```
    @Value("${datasource.url}")
```

```
    private String dbUrl;
```

```
    public void connect(@Value("${datasource.login}") String pLogin) {
```

```
        // ...
```

```
    }
```

```
}
```

➤ Le fichier properties chargé dans le contexte Spring :

```
datasource.url=jdbc:mysql://localhost/banque?useSSL=false
```

```
datasource.login=root
```


Faire cohabiter les différents styles de configuration

- Une application Spring peut faire cohabiter les trois modes d'IoC
 - XML ou @Configuration
 - ❑ Aucune limite technique mais un peu lourd.
 - Annotation
 - ❑ Seule la classe que l'on code peut être annotée.

Faire cohabiter les différents styles de configuration

Bonne Pratique

- Il est important de laisser le Spring gérer **nos** beans (ceux que l'on a codé)
 - Eléments à gérer par Stéréotypes et @Autowired :
 - ❑ DAOs, Services, Contrôleurs
- Pour le reste, parties techniques, il est préférable d'avoir une configuration (XML, @Configuration, Spring Boot)
 - Eléments à NE PAS gérer par Stéréotypes
 - ❑ DataSource, annuaires, paramétrages techniques



Travaux pratiques

27

Réaliser les travaux pratiques suivants:

TP 08 : Un projet complet en annotations